

MIT-STYLE OPEN CURRICULUM

HTML & CSS

The Complete Developer Course

*From Zero to Advanced — Flexbox · Grid · Animations · Selectors ·
Typography*

■■■ Course Author

Prof. Claude · MIT CSAIL

Web Technologies · Computer Science

■ 8 Modules

■ All Syntax Explained

■ Flexbox & Grid Deep Dive

■ CSS Animations & Transitions

01 HTML Foundations

02 HTML Deep Dive

03 CSS Foundations

04 The Box Model & Layout

05 Typography & Colors

06 Flexbox

07 CSS Grid

08 Transitions & Animations

Table of Contents

MODULE 01 — HTML Foundations

- › What is HTML?
- › Document Structure & DOCTYPE
- › Head vs Body
- › Essential Meta Tags
- › Semantic vs Non-Semantic Elements

MODULE 02 — HTML Deep Dive

- › All Text Tags & Headings
- › Links & Anchors
- › Images & Attributes
- › Lists (ol, ul, dl)
- › Tables
- › Forms & Input Types
- › HTML5 Semantic Elements
- › Media Elements
- › Inline vs Block Elements
- › Global Attributes

MODULE 03 — CSS Foundations

- › How CSS Works — Cascade & Specificity
- › Selectors: Type, Class, ID, Attribute, Pseudo
- › Combinators
- › The Cascade & Inheritance
- › Units: px, em, rem, vw, vh, %, fr
- › How to Link CSS
- › CSS Variables (Custom Properties)

MODULE 04 — The Box Model & Layout

- › Content, Padding, Border, Margin
- › box-sizing: border-box
- › Display Property
- › Position: static, relative, absolute, fixed, sticky
- › z-index & Stacking Contexts
- › Overflow
- › Float & Clear (legacy)

MODULE 05 — Typography & Colors

- › Font Properties
- › Text Properties
- › Web Fonts & @font-face
- › Color Formats
- › Backgrounds

- › Shadows
- › Gradients
- › Opacity & Filters

MODULE 06 — Flexbox

- › The Flex Container
- › Flex Direction & Wrap
- › justify-content
- › align-items & align-content
- › Flex Children: grow, shrink, basis
- › order & align-self
- › Common Flexbox Patterns

MODULE 07 — CSS Grid

- › Grid Container Setup
- › Defining Rows & Columns
- › fr unit & repeat()
- › grid-gap / gap
- › Placing Items: lines & areas
- › grid-template-areas
- › auto-fill & auto-fit
- › Nested Grids
- › Grid vs Flexbox — When to Use Which

MODULE 08 — Transitions & Animations

- › transition Property
- › Timing Functions
- › @keyframes Rule
- › animation Property
- › transform: translate, rotate, scale, skew
- › transform-origin
- › 3D Transforms
- › will-change & Performance
- › Responsive Design & Media Queries
- › Pseudo-elements ::before & ::after

HTML Foundations

Understand what HTML is, how browsers parse it, and how to structure every document correctly.

1.1 What is HTML?

HTML — HyperText Markup Language — is the skeleton of every webpage. It is not a programming language; it is a *markup language*. You wrap content in **elements** (tags) to tell the browser what kind of content it is — a paragraph, a heading, a link, an image. The browser reads your HTML, builds a tree called the **DOM (Document Object Model)**, and renders it visually.

■ Think of HTML as the bones, CSS as the skin and clothes, and JavaScript as the muscles. You must understand the bones before anything else.

1.2 Anatomy of an HTML Element

Every HTML element follows a consistent pattern:

```
<tagname attribute="value"> Content goes here </tagname>

<!-- Example: -->
<p class="intro"> Hello, world! </p>

<!--
Opening tag : <p class="intro">
Attribute : class="intro"
Content : Hello, world!
Closing tag : </p>
-->
```

Some elements are **void elements** — they have no content and no closing tag:

```

<br>
<hr>
<input type="text">
<meta charset="UTF-8">
<link rel="stylesheet" href="style.css">
```

1.3 The Complete Document Structure

```
<!DOCTYPE html>
<html lang="en">

<head>
  <!-- Metadata – invisible to users, vital for browsers & SEO -->
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <meta name="description" content="Page description for search engines">
  <title>Page Title (appears in browser tab)</title>
  <link rel="stylesheet" href="style.css">
</head>

<body>
  <!-- All visible content lives here -->
  <h1>Hello World</h1>
  <p>First paragraph.</p>
  <script src="app.js"></script> <!-- JS usually at end of body -->
</body>

</html>
```

Each line explained:

Property	Values	What it does
<code><!DOCTYPE html></code>	<i>Any string</i>	Tells browser: use modern HTML5 rules. Always first line. Not a tag — a declaration.

<code><html lang="en"></code>	<i>en, fr, hi, ar ...</i>	Root element wrapping everything. lang= helps screen readers & SEO.
<code><head></code>	—	Container for metadata — nothing in here renders on screen.
<code><meta charset></code>	<i>UTF-8</i>	Character encoding. UTF-8 supports every language. Always include.
<code><meta name=viewport></code>	<i>see content=</i>	Critical for mobile. Tells phone browsers not to zoom out to desktop width.
<code><title></code>	<i>Any text</i>	Text shown in browser tab and search results.
<code><link rel=stylesheet></code>	<i>href= path</i>	Attaches a CSS file to this page.
<code><body></code>	—	Everything the user sees lives inside <body>.

1.4 Comments in HTML

```

<!-- This is a single-line comment -->

<!--
This is a
multi-line comment.
The browser ignores everything between these markers.
-->

```

■ Comments are developer notes. They never appear on screen. Use them to explain complex sections or temporarily disable code.

1.5 Nesting Rules

Elements must be properly nested — the last opened tag must be the first closed. This is the most common beginner mistake:


```
<!-- ■ CORRECT nesting -->
<p>This is <strong>bold <em>and italic</em></strong> text.</p>

<!-- ■ INCORRECT nesting – tags overlap -->
<p>This is <strong>bold <em>and italic</strong> text.</em></p>
```

1.6 Semantic vs Non-Semantic Elements

A **semantic** element clearly describes its meaning to both the browser and the developer. A **non-semantic** element says nothing about its content.

```
<!-- Non-semantic: <div> and <span> mean nothing by themselves -->
<div class="nav">...</div>
<span class="highlight">word</span>

<!-- Semantic: the tag name describes the role -->
<nav>...</nav>
<header>...</header>
<main>...</main>
<article>...</article>
<aside>...</aside>
<footer>...</footer>
<mark>word</mark>
```

■ Always prefer semantic HTML. It helps screen readers, search engines, and other developers understand your page structure without reading the CSS.

HTML Deep Dive

Every important HTML element, its syntax, purpose, and when to use it.

2.1 Headings

There are six heading levels, **h1** through **h6**. They communicate document hierarchy — not just visual size.

```
<h1>Main Page Title</h1> <!-- Only ONE per page -->
<h2>Major Section</h2> <!-- Multiple allowed -->
<h3>Sub-section</h3>
<h4>Sub-sub-section</h4>
<h5>Rarely needed</h5>
<h6>Almost never needed</h6>
```

■ Never skip heading levels (e.g., h1 → h3) for visual size. Use CSS for sizing. Screen readers navigate pages by heading structure.

2.2 Paragraph & Inline Text Elements

```
<p>A paragraph. Browser adds top/bottom margin automatically.</p>

<strong>Bold – semantic importance</strong>
<b>Bold – visual only, no semantic meaning</b>

<em>Italic – semantic emphasis</em>
<i>Italic – visual only (titles, foreign words, technical terms)</i>

<u>Underline</u>

<s>Strikethrough</s>

<del>Deleted text</del> <!-- semantic deletion -->
<ins>Inserted text</ins> <!-- semantic insertion -->

<mark>Highlighted text</mark>

<small>Fine print / legal text</small>

<sub>Subscript: H<sub>2</sub>O</sub>
<sup>Superscript: E=mc<sup>2</sup></sup></sup>

<code>Inline code snippet</code>

<kbd>Keyboard input: press <kbd>Ctrl+C</kbd></kbd>

<pre>Preserves whitespace
and line breaks exactly as written</pre>

<blockquote cite="https://source.com">A block quotation</blockquote>

<q>An inline quotation – browser adds quotation marks</q>

<abbr title="HyperText Markup Language">HTML</abbr>

<cite>A Book Title or Creative Work</cite>

<time datetime="2024-01-15">January 15, 2024</time>

<br> <!-- Line break – use sparingly, never for spacing -->
<hr> <!-- Thematic break – horizontal rule -->
<wbr> <!-- Word break opportunity hint for long strings -->
```

2.3 Links — <a>

The anchor tag creates hyperlinks. The **href** attribute is the destination.

```
<!-- External link -->
<a href="https://example.com">Visit Example</a>

<!-- Opens in new tab (_blank always pair with rel for security) -->
<a href="https://example.com" target="_blank" rel="noopener noreferrer">
Open in new tab
</a>

<!-- Internal link (same site, relative path) -->
<a href="/about.html">About Us</a>
<a href="../index.html">Go up one folder</a>

<!-- Anchor link (jump to ID on same page) -->
<a href="#section2">Jump to Section 2</a>
<section id="section2">...</section>

<!-- Email link -->
<a href="mailto:hello@example.com">Send Email</a>

<!-- Phone link (mobile) -->
<a href="tel:+919876543210">Call Us</a>

<!-- Download link -->
<a href="/files/report.pdf" download="annual-report.pdf">Download PDF</a>
```

2.4 Images —

```

<!-- Basic image -->


<!-- alt is NOT optional: screen readers read it, shows when image fails -->
<!-- Decorative images: alt="" (empty, not absent) -->


<!-- Width and height SHOULD be set to prevent layout shift (CLS) -->


<!-- loading=lazy defers off-screen images -->


<!-- Responsive image with srcset -->


<!-- Figure with caption -->
<figure>

<figcaption>Fig 1. High-level system overview.</figcaption>
</figure>

```

2.5 Lists

```

<!-- Unordered list (bullets) -->
<ul>
<li>First item</li>
<li>Second item</li>
<li>
  Nested list inside an li:
  <ul>
    <li>Sub-item A</li>
    <li>Sub-item B</li>
  </ul>
</li>
</ul>

<!-- Ordered list (numbers) -->
<ol>
<li>First step</li>
<li>Second step</li>
</ol>

<!-- Ordered list attributes -->
<ol type="A"> <!-- A, B, C ... -->
<ol type="i"> <!-- i, ii, iii ... -->
<ol start="5"> <!-- starts numbering at 5 -->
<ol reversed> <!-- counts down -->

<!-- Description list (term + definition) -->
<dl>
<dt>HTML</dt>
<dd>HyperText Markup Language – the structure of web pages.</dd>
<dt>CSS</dt>
<dd>Cascading Style Sheets – the presentation layer.</dd>
</dl>

```

2.6 Tables

Tables are for **tabular data** only — never for layout. A complete, accessible table uses semantic elements:

```

<table>
<caption>Monthly Sales by Region</caption> <!-- title above table -->

<thead> <!-- header rows -->
<tr>
<th scope="col">Region</th>
<th scope="col">Q1</th>
<th scope="col">Q2</th>
</tr>
</thead>

<tbody> <!-- data rows -->
<tr>
<th scope="row">North</th>
<td>$12,000</td>
<td>$15,000</td>
</tr>
<tr>
<th scope="row">South</th>
<td>$8,000</td>
<td>$9,500</td>
</tr>
</tbody>

<tfoot> <!-- summary row -->
<tr>
<th scope="row">Total</th>
<td>$20,000</td>
<td>$24,500</td>
</tr>
</tfoot>
</table>

<!-- Spanning multiple columns or rows -->
<td colspan="2">Spans 2 columns</td>

<td rowspan="3">Spans 3 rows</td>

```

2.7 Forms & Inputs

Forms collect user data. The **action** attribute is the server endpoint; **method** is GET or POST.

```
<form action="/submit" method="POST" novalidate>
<!-- label MUST be associated with its input -->
<label for="username">Username</label>
<input type="text"
id="username"
name="username"
placeholder="Enter username"
required
minlength="3"
maxlength="20"
autocomplete="username">

<!-- Password -->
<input type="password" id="pwd" name="pwd" required>

<!-- Email – browser validates format -->
<input type="email" id="email" name="email">

<!-- Number -->
<input type="number" min="1" max="100" step="1" value="1">

<!-- Range slider -->
<input type="range" min="0" max="100" value="50">

<!-- Date, time, datetime-local -->
<input type="date">
<input type="time">
<input type="datetime-local">

<!-- Checkbox -->
<input type="checkbox" id="agree" name="agree" value="yes">
<label for="agree">I agree to the terms</label>

<!-- Radio buttons (same name = one group) -->
```

```

<input type="radio" id="male" name="gender" value="male">
<label for="male">Male</label>
<input type="radio" id="female" name="gender" value="female">
<label for="female">Female</label>

<!-- Select dropdown -->
<select name="country" id="country">
<option value="">-- Select country --</option>
<optgroup label="Asia">
<option value="in">India</option>
<option value="jp">Japan</option>
</optgroup>
<option value="us" selected>United States</option>
</select>

<!-- Multi-line text -->
<textarea name="message" id="message" rows="5" cols="40"
placeholder="Your message..."></textarea>

<!-- File upload -->
<input type="file" accept=".pdf,.jpg,.png" multiple>

<!-- Hidden field -->
<input type="hidden" name="csrf_token" value="abc123">

<!-- Buttons -->
<button type="submit">Submit Form</button>
<button type="reset">Reset</button>
<button type="button">Custom JS action</button>

<!-- Group related fields -->
<fieldset>
<legend>Shipping Address</legend>
<input type="text" name="street" placeholder="Street">
<input type="text" name="city" placeholder="City">

</fieldset>

</form>

```

2.8 HTML5 Semantic Layout Elements

```
<header> Site/section header, logo, main nav</header>
<nav> Navigation links</nav>
<main> The primary unique content of the page (ONE per page)</main>
<article> Self-contained content (blog post, news story, comment)</article>
<section> Thematic grouping of content with a heading</section>
<aside> Tangentially related content (sidebar, callout box)</aside>
<footer> Footer of a page or section</footer>
<address> Contact information</address>

<!-- Typical page structure -->
<body>
<header>
<nav>...</nav>
</header>
<main>
<article>
<header><h1>Post Title</h1></header>
<section>...</section>
<section>...</section>
<footer><address>Author info</address></footer>
</article>
<aside>Related posts</aside>
</main>
<footer>Site footer</footer>
</body>
```

2.9 Media Elements

```

<!-- VIDEO -->
<video
src="movie.mp4"
width="720"
height="405"
controls <!-- show play/pause/volume controls -->
autoplay muted <!-- autoplay requires muted in most browsers -->
loop
poster="thumbnail.jpg" <!-- shown before video plays -->
preload="metadata" <!-- none | metadata | auto -->
>

<!-- Fallback for multiple formats -->
<source src="movie.mp4" type="video/mp4">
<source src="movie.webm" type="video/webm">
<p>Your browser does not support video.</p>
</video>

<!-- AUDIO -->
<audio controls>
<source src="podcast.mp3" type="audio/mpeg">
<source src="podcast.ogg" type="audio/ogg">
</audio>

<!-- IFRAME – embed external content -->
<iframe
src="https://www.youtube.com/embed/VIDEO_ID"
width="560"
height="315"
title="YouTube video player" <!-- required for accessibility -->
frameborder="0"
allowfullscreen
loading="lazy"
></iframe>

```

2.10 Global Attributes

These attributes can be placed on *any* HTML element:

Property	Values	What it does
id	<i>Unique string</i>	Unique identifier on the page. Used by CSS (#id), JS (getElementById), and anchor links (#id).
class	<i>Space-separated names</i>	Assigns one or more class names. Used by CSS (.class) and JS. Reusable across many elements.
style	<i>CSS declarations</i>	Inline CSS. Avoid for anything non-trivial — hard to maintain and override.
title	<i>Any text</i>	Tooltip shown on hover. Also used by screen readers as supplemental info.
lang	<i>BCP 47 code</i>	Language of the element's content. Inherits from <html lang=>. Override per-element if needed.
dir	<i>ltr rtl auto</i>	Text direction. ltr=left-to-right, rtl=right-to-left (Arabic, Hebrew, Urdu).
hidden	<i>boolean</i>	Hides the element (equivalent to display:none). Semantic: content is not relevant right now.
tabindex	<i>integer</i>	Controls keyboard tab order. 0=natural order, -1=focusable by JS only, positive=explicit order.
contenteditable	<i>true false</i>	Makes element editable in browser. Foundation for rich text editors.
data-*	<i>Any string</i>	Custom data attributes. Store extra data for JS without using non-standard attributes.
aria-*	<i>Various</i>	Accessibility attributes. aria-label, aria-hidden, aria-expanded etc. for screen readers.
draggable	<i>true false</i>	Enables drag-and-drop HTML5 API.

■ data-* attributes are your best friends for storing component state: . Access in JS:
el.dataset.userId, el.dataset.role

CSS Foundations

The cascade, specificity, selectors, units, and variables — the engine room of CSS.

3.1 How to Apply CSS — Three Methods

```
<!-- 1. External stylesheet (BEST – reusable, cacheable) -->
<link rel="stylesheet" href="style.css">

<!-- 2. Internal <style> block (OK for single-page or overrides) -->
<style>
body { background: #fff; }
</style>

<!-- 3. Inline style (AVOID – hardest to maintain & override) -->
<p style="color: red; font-size: 16px;">Text</p>
```

3.2 CSS Rule Anatomy

```
selector {
  property: value; /* declaration */,
  property: value;
}

/* Example */
h1 {
  color: #0A1628;
  font-size: 2rem;
  margin-bottom: 1em;
}
```


3.3 Selectors — Complete Reference

Basic Selectors

```
/* Type selector – targets all <p> elements */
p { color: black; }

/* Class selector – targets elements with class="btn" */
.btn { background: blue; }

/* ID selector – targets element with id="header" */
#header { height: 80px; }

/* Universal selector – targets every element */
* { box-sizing: border-box; }

/* Attribute selectors */
[type="text"] { border: 1px solid gray; } /* exact match */
[href^="https"] { color: green; } /* starts with */
[href$=".pdf"] { color: red; } /* ends with */
[class*="btn"] { cursor: pointer; } /* contains */
[lang|="en"] { } /* en or en-US, en-GB ... */
[data-status~="active"] { } /* space-separated word */
```

Combinators

```
/* Descendant – any <a> inside .nav (any depth) */
.nav a { text-decoration: none; }

/* Child – only DIRECT children */
ul > li { list-style: none; }

/* Adjacent sibling – first <p> IMMEDIATELY after <h2> */
h2 + p { font-size: 1.1em; }

/* General sibling – all <p> after <h2> at the same level */
h2 ~ p { color: gray; }
```

Pseudo-Classes

```

/* User interaction */
a:hover { color: blue; } /* mouse over */
a:focus { outline: 2px solid blue; } /* keyboard focus */
a:active { color: red; } /* being clicked */
a:visited { color: purple; }

/* Form states */
input:focus { border-color: blue; }
input:disabled { opacity: 0.5; }
input:checked { accent-color: green; }
input:required { border-color: orange; }
input:valid { border-color: green; }
input:invalid { border-color: red; }
input:placeholder-shown { font-style: italic; }

/* Structural */
li:first-child { font-weight: bold; }
li:last-child { border-bottom: none; }
li:nth-child(2) { background: #eee; } /* exactly 2nd */
li:nth-child(odd) { background: #f5f5f5; }
li:nth-child(even) { background: white; }
li:nth-child(3n) { color: red; } /* every 3rd: 3,6,9,... */
li:nth-child(3n+1) { } /* 1,4,7,... */
li:nth-last-child(1) { } /* from the end */

/* Type-specific */
p:first-of-type { font-size: 1.2em; }
p:last-of-type { margin-bottom: 0; }
p:nth-of-type(2) { color: gray; }

/* :not – negation */
li:not(:last-child) { border-bottom: 1px solid #ddd; }
button:not(:disabled) { cursor: pointer; }

/* :is() and :where() – group selectors efficiently */

:is(h1,h2,h3) { font-family: Georgia; }
:where(header, footer) a { color: white; }

/* :has() – parent selector (modern CSS!) */
label:has(+ input:required)::after { content: " *"; color: red; }
figure:has(figcaption) { border: 1px solid #ddd; }

```

Pseudo-Elements

```
/* ::before and ::after – inject content before/after element */
h2::before { content: "$ "; color: gray; }
a[href^="https"]::after { content: " ■"; font-size: 0.8em; }

/* Always requires content: "" even if empty */
.icon::before { content: ""; display: block; width: 20px; height: 20px; }

/* ::first-line, ::first-letter */
p::first-line { font-variant: small-caps; }
p::first-letter { font-size: 3em; float: left; }

/* ::selection – text the user has highlighted */
::selection { background: #FBBF24; color: black; }

/* ::placeholder – input placeholder text */
input::placeholder { color: #aaa; font-style: italic; }
```

3.4 The Cascade & Specificity

When two rules target the same element and same property, CSS uses three criteria in order: **1. Origin & Importance**, **2. Specificity**, **3. Source Order**.

Specificity Scoring

Think of specificity as a 4-column score: (Inline, ID, Class/Attr/Pseudo-class, Type/Pseudo-element). Higher score wins.

```

/* Score: (0, 0, 0, 1) – type */
p { color: black; }

/* Score: (0, 0, 1, 0) – class */
.intro { color: blue; }

/* Score: (0, 1, 0, 0) – ID */
#hero { color: green; }

/* Score: (1, 0, 0, 0) – inline style – beats all above */
<p style="color: red">

/* Score: (0, 0, 1, 1) – class + type */
p.intro { color: purple; }

/* Score: (0, 1, 1, 0) – ID + class */
#hero .title { color: orange; }

/* !important – nuclear option, overrides everything */
p { color: pink !important; } /* avoid in your own code */

```

■ Rule of thumb: use classes for everything. Avoid IDs for styling. Never use !important unless overriding third-party styles you can't change.

3.5 Inheritance

Some CSS properties are **inherited** by children from their parent; others are not.

```

/* Inherited by default (text-related mostly) */
/* color, font-family, font-size, font-weight, font-style, */
/* line-height, letter-spacing, text-align, list-style, cursor, visibility */
/* NOT inherited (box/layout-related) */
/* margin, padding, border, background, width, height, */
/* position, display, overflow */
/* Explicit control keywords */
.child {
color: inherit; /* force inheritance */
border: inherit; /* force even non-inherited props */
color: initial; /* reset to browser default */
color: unset; /* inherit if inheritable, else initial */
color: revert; /* reset to browser stylesheet default */
}

```

3.6 CSS Units — Complete Guide

Absolute Units

Property	Values	What it does
px	<i>1px = 1 device-independent pixel</i>	Most common. Consistent across devices. Use for borders, shadows, fixed-size elements.
pt	<i>1pt = 1/72 inch</i>	For print stylesheets. Not for screens.
cm / mm / in	<i>Physical measurements</i>	For print only.

Relative Units

Property	Values	What it does
em	<i>Relative to the element's own font-size</i>	Compounds: a child inside a parent both using em will multiply. 1.5em on a 16px element = 24px. Great for padding/margin relative to text size.

rem	<i>Relative to the root (<html>) font-size</i>	Never compounds. If <code>html { font-size: 16px }</code> , <code>1rem</code> = always 16px anywhere. Best for font sizes & spacing.
vw	<i>1% of the viewport width</i>	<code>100vw</code> = full width. Use for full-width sections. Beware: includes scrollbar width.
vh	<i>1% of the viewport height</i>	<code>100vh</code> = full height. Use for full-screen sections. Known issue on mobile browsers (toolbar changes height).
svh/dvh/lvh	<i>Small/dynamic/large viewport height</i>	Modern fix for mobile. <code>dvh</code> adjusts dynamically. Use <code>dvh</code> for full-screen mobile apps.
%	<i>Percentage of parent element's corresponding value</i>	width: <code>50%</code> = half of parent's width. margin: <code>10%</code> = 10% of parent's width.
fr	<i>Fraction of available space in Grid</i>	Only valid inside grid-template-columns/rows. <code>1fr 2fr</code> = 1/3 and 2/3 of space.
ch	<i>Width of the '0' character</i>	width: <code>60ch</code> keeps lines readable (~60 characters wide). Great for prose.
ex	<i>Height of lowercase 'x'</i>	Rare. For precise typographic fine-tuning.
min / max / clamp	<i>CSS functions</i>	<code>clamp(min, preferred, max)</code> for fluid type. <code>font-size: clamp(1rem, 2vw, 1.5rem)</code>

3.7 CSS Custom Properties (Variables)

```

/* Declare in :root for global scope */
:root {
  --color-primary: #1A56DB;
  --color-text: #0A1628;
  --font-body: "Inter", sans-serif;
  --spacing-sm: 8px;
  --spacing-md: 16px;
  --spacing-lg: 32px;
  --border-radius: 6px;
}

/* Use with var() */
button {
  background: var(--color-primary);
  color: white;
  padding: var(--spacing-sm) var(--spacing-md);
  border-radius: var(--border-radius);
}

/* Fallback value (second argument) */
.card {
  background: var(--card-bg, white); /* white if --card-bg not set */
}

/* Override locally (scoped to a component) */
.dark-theme {
  --color-primary: #60A5FA;
  --color-text: #F1F5F9;
}

/* Change with JavaScript */
document.documentElement.style.setProperty('--color-primary', '#ff0000');

```

■ CSS variables are live — changing them in JS instantly updates every element using them. This is how theme switchers, design tokens, and dynamic color systems work.

The Box Model & Layout

Every element is a box. Master the model, display modes, and positioning.

4.1 The CSS Box Model

Every HTML element renders as a rectangular box with four layers from inside out: **Content** → **Padding** → **Border** → **Margin**.

```

.box {
  /* CONTENT: the actual text or child elements */
  width: 300px;
  height: 200px;

  /* PADDING: space INSIDE the border, part of the background */
  padding: 20px; /* all sides */
  padding: 10px 20px; /* top/bottom left/right */
  padding: 5px 10px 15px 20px; /* top right bottom left (clockwise) */
  padding-top: 10px; /* individual sides */

  /* BORDER: wraps padding */
  border: 2px solid #333;
  border-width: 1px 2px 1px 2px;
  border-style: solid; /* solid | dashed | dotted | double | none */
  border-color: #333;
  border-radius: 8px; /* round corners */
  border-radius: 50%; /* circle */
  border-top: 3px dashed red; /* individual sides */

  /* MARGIN: space OUTSIDE the border, transparent */
  margin: 20px;
  margin: 10px auto; /* auto centers a block element horizontally */
  margin-top: -10px; /* negative margins allowed! */
}

```

box-sizing — The Most Important CSS Property

```

/* DEFAULT (content-box): width = content only */
/* Adding padding/border INCREASES total size – confusing! */
.box-default {
width: 300px;
padding: 20px;
border: 2px solid black;
/* Total width = 300 + 20+20 + 2+2 = 344px */
}

/* border-box: width = content + padding + border (PREDICTABLE) */
.box-border {
box-sizing: border-box;
width: 300px;
padding: 20px;
border: 2px solid black;
/* Total width = exactly 300px. Content shrinks to 256px. */
}

/* Apply globally – always do this */
*, *::before, *::after {
box-sizing: border-box;
}

```

■ Always put `box-sizing: border-box` in your CSS reset. Every professional codebase does this. It makes layout math intuitive.

Margin Collapse

Vertical margins between adjacent block elements *collapse* — the larger margin wins instead of adding up. This is intentional and affects layout significantly.

```

h2 { margin-bottom: 24px; }
p { margin-top: 16px; }
/* Space between h2 and p = 24px, NOT 40px — they collapse */
/* Collapse does NOT happen: */
/* 1. When a parent has padding or border separating the margins */
/* 2. Between flex or grid children */
/* 3. For floated or absolutely positioned elements */
/* 4. When using overflow:hidden on the parent */

```

4.2 The display Property

Property	Values	What it does
block	<i>div, p, h1-h6, section...</i>	Takes full width, starts on new line. Width/height/margin work fully.
inline	<i>span, a, strong, em...</i>	Flows with text. Width/height have NO effect. Top/bottom margin ignored.
inline-block	<i>img, button, input</i>	Flows with text BUT width/height/margin fully work. Best of both.
none	<i>Any</i>	Removes from layout entirely — as if it doesn't exist. Space is not reserved.
flex	<i>Any</i>	Creates a flex formatting context for direct children. Module 6 covers this fully.
inline-flex	<i>Any</i>	Like flex but the container itself is inline.
grid	<i>Any</i>	Creates a grid formatting context. Module 7 covers this fully.
inline-grid	<i>Any</i>	Like grid but the container itself is inline.
contents	<i>Any</i>	Element itself vanishes; its children participate in the parent's layout.
table/table-cell	<i>Any</i>	Makes non-table elements behave like table elements. Rarely needed today.

<code>list-item</code>	<code>li</code>	Adds the list bullet. li elements have this by default.
------------------------	-----------------	---

4.3 The position Property

Positioning takes elements out of (or modifies) the normal document flow. Combined with **top**, **right**, **bottom**, **left** and **z-index**.

Property	Values	What it does
<code>static</code>	<i>Default</i>	Normal flow. top/right/bottom/left/z-index have NO effect.
<code>relative</code>	<i>Normal flow + offset</i>	Stays in normal flow but moves visually. Offset is relative to its own original position. Creates a positioning context for absolute children.
<code>absolute</code>	<i>Removed from flow</i>	Positioned relative to the nearest ancestor with position != static. If none, relative to the initial containing block (viewport). Other elements fill the gap.
<code>fixed</code>	<i>Removed from flow</i>	Like absolute but relative to the viewport always. Stays on screen when scrolling. Used for sticky navbars, modals.
<code>sticky</code>	<i>Hybrid: relative + fixed</i>	Behaves as relative until it reaches a threshold (e.g., top: 0), then becomes fixed within its scroll container. Used for sticky table headers.

```
/* Absolute centering pattern */
.parent {
position: relative; /* creates positioning context */
height: 400px;
}
.centered {
position: absolute;
top: 50%;
left: 50%;
transform: translate(-50%, -50%); /* account for element's own size */
}

/* Sticky header */
header {
position: sticky;
top: 0; /* sticks at 0px from viewport top */
z-index: 100;
}

/* Cover the full viewport with a modal overlay */
.overlay {
position: fixed;
inset: 0; /* shorthand for top:0; right:0; bottom:0; left:0 */
background: rgba(0,0,0,0.5);
z-index: 999;
}
```

4.4 z-index & Stacking Contexts

```
/* z-index only works on positioned elements (not static) */
.modal { position: fixed; z-index: 1000; }
.tooltip { position: absolute; z-index: 500; }
.header { position: sticky; z-index: 100; }

/* A stacking context is created by: */
/* position:relative/absolute/fixed + z-index != auto */
/* opacity < 1 */
/* transform, filter, clip-path (any value) */
/* isolation: isolate */

/* Use isolation: isolate to create a new stacking context */
/* WITHOUT changing z-index – great for component isolation */
.component {
  isolation: isolate;
}
```

4.5 overflow

```
.card {
  overflow: hidden; /* clip overflowing images/content */
  overflow-x: auto; /* horizontal scroll */
  overflow-y: hidden; /* clip vertical */
  overflow: auto; /* scroll when needed */
}

/* Prevent text overflow with ellipsis */
.truncate {
  white-space: nowrap;
  overflow: hidden;
  text-overflow: ellipsis;
}
```


Typography & Colors

Font properties, text control, color formats, backgrounds, gradients, and shadows.

5.1 Font Properties

```
p {  
  /* Font family – always list fallbacks */  
  font-family: "Inter", "Helvetica Neue", Arial, sans-serif;  
  /* Categories: serif | sans-serif | monospace | cursive | fantasy | system-ui */  
  /* Font size */  
  font-size: 16px; /* absolute */  
  font-size: 1rem; /* relative to root – PREFERRED */  
  font-size: 1.25em; /* relative to parent */  
  font-size: clamp(1rem, 2vw + 0.5rem, 1.5rem); /* fluid */  
  /* Weight */  
  font-weight: normal; /* 400 */  
  font-weight: bold; /* 700 */  
  font-weight: 100; /* thin – 100 to 900 in steps of 100 */  
  font-weight: 600; /* semi-bold */  
  /* Style */  
  font-style: normal | italic | oblique;  
  /* Shorthand (style weight size/line-height family) */  
  font: italic 600 1rem/1.5 "Inter", sans-serif;  
}
```

5.2 Text Properties

```
p {
  /* Alignment */
  text-align: left | center | right | justify | start | end;

  /* Line height (unitless preferred – multiplied by font-size) */
  line-height: 1.6; /* 1.5–1.8 for body text */
  line-height: 1; /* tight, for headings */

  /* Spacing */
  letter-spacing: 0.05em; /* tracking */
  letter-spacing: -0.02em; /* tight (for headings) */
  word-spacing: 0.1em;

  /* Decoration */
  text-decoration: none;
  text-decoration: underline;
  text-decoration: underline dotted red 2px; /* shorthand */
  text-decoration-line: underline | overline | line-through;
  text-decoration-style: solid | dashed | dotted | double | wavy;
  text-decoration-color: blue;
  text-decoration-thickness: 2px;
  text-underline-offset: 4px; /* space between text & underline */

  /* Transform */
  text-transform: none | uppercase | lowercase | capitalize;

  /* Indent */
  text-indent: 2em;

  /* White space handling */
  white-space: normal; /* wrap, collapse whitespace */
  white-space: nowrap; /* never wrap */
  white-space: pre; /* preserve all whitespace, no wrap */
  white-space: pre-wrap; /* preserve, but wrap */
  white-space: pre-line; /* preserve newlines, collapse spaces */
}
```

```

/* Word/overflow break */
overflow-wrap: break-word; /* break long unbreakable strings */
word-break: break-all;
hyphens: auto; /* browser adds hyphens at breaks (needs lang=) */

/* Writing mode */
writing-mode: horizontal-tb | vertical-rl | vertical-lr;
text-orientation: mixed | upright | sideways;
}

```

5.3 Web Fonts & @font-face

```

/* Google Fonts – simplest approach (import in CSS) */
@import url("https://fonts.googleapis.com/css2?family=Inter:wght@400;600;700&display=swap");

/* Self-hosted with @font-face */
@font-face {
  font-family: "Inter";
  src: url("fonts/inter.woff2") format("woff2"),
  url("fonts/inter.woff") format("woff");
  font-weight: 400;
  font-style: normal;
  font-display: swap; /* show fallback immediately, swap when loaded */
}

/* Variable fonts – one file, all weights */
@font-face {
  font-family: "InterVariable";
  src: url("inter-variable.woff2") format("woff2-variations");
  font-weight: 100 900; /* range supported */
}

body {
  font-family: "Inter", system-ui, sans-serif;
  font-optical-sizing: auto; /* for variable fonts */
}

```

5.4 Color Formats

```
/* Named colors (147 available) */
color: red; color: tomato; color: cornflowerblue;

/* Hexadecimal – #RRGGBB */
color: #FF5733; /* full 6-digit */
color: #F53; /* shorthand for #FF5533 */
color: #FF5733AA; /* 8-digit: last 2 = alpha (AA = ~67% opaque) */

/* RGB */
color: rgb(255, 87, 51);
color: rgb(255 87 51); /* modern space syntax */
color: rgba(255, 87, 51, 0.8); /* 0.8 = 80% opaque */
color: rgb(255 87 51 / 0.8); /* modern syntax */

/* HSL – Hue Saturation Lightness (most intuitive for adjusting) */
color: hsl(14, 100%, 60%); /* hue 14° = orange-red */
color: hsl(14deg 100% 60%); /* deg keyword optional */
color: hsl(14 100% 60% / 0.8); /* with alpha */

/* HSL mental model: */
/* Hue: 0/360=red, 60=yellow, 120=green, 180=cyan, 240=blue, 300=magenta */
/* Saturation: 0%=gray, 100%=vivid */
/* Lightness: 0%=black, 50%=normal, 100%=white */

/* LCH / OKLCH – modern, perceptually uniform (CSS Color 4) */
color: oklch(65% 0.2 30); /* lightness chroma hue – best for color mixing */
```

5.5 Backgrounds

```

.box {
  /* Solid color */
  background-color: #f0f4ff;

  /* Image */
  background-image: url("bg.jpg");

  /* Position: keyword | % | px */
  background-position: center center;
  background-position: 50% 50%;
  background-position: right 20px top 10px; /* offset from edge */

  /* Repeat */
  background-repeat: no-repeat | repeat | repeat-x | repeat-y | space | round;

  /* Size */
  background-size: cover; /* fill container, may crop */
  background-size: contain; /* fit inside, may letterbox */
  background-size: 200px 100px;
  background-size: 50% auto;

  /* Attachment */
  background-attachment: scroll; /* default – moves with element */
  background-attachment: fixed; /* fixed to viewport = parallax effect */
  background-attachment: local; /* moves with element content when scrolling */

  /* Origin & clip */
  background-origin: padding-box | border-box | content-box;
  background-clip: padding-box | border-box | content-box | text;

  /* Multiple backgrounds (comma-separated, first = on top) */
  background:
    url("overlay.png") center/cover no-repeat,
    url("texture.jpg") repeat,
    #1a1a2e;
}

```

5.6 Gradients

```

/* Linear gradient */
background: linear-gradient(to right, #667eea, #764ba2);
background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
background: linear-gradient(to bottom, #fff 0%, #fff 50%, #000 50%, #000 100%);

/* Radial gradient */
background: radial-gradient(circle, #667eea, #764ba2);
background: radial-gradient(ellipse at top left, #ff0080, transparent);

/* Conic gradient (pie charts, color wheels) */
background: conic-gradient(red 0deg, yellow 90deg, green 180deg, blue 270deg);

/* Repeating gradients */
background: repeating-linear-gradient(
  45deg,
  #f1f1f1 0px,
  #f1f1f1 10px,
  white 10px,
  white 20px
);

/* Gradient text */
h1 {
background: linear-gradient(135deg, #667eea, #764ba2);
-webkit-background-clip: text;
background-clip: text;
color: transparent;
}

```

5.7 Shadows

```

/* box-shadow: offset-x offset-y blur spread color */
.card {
box-shadow: 0 4px 6px rgba(0, 0, 0, 0.1);
box-shadow: 0 1px 3px rgba(0,0,0,0.12), 0 1px 2px rgba(0,0,0,0.24); /* multiple */
box-shadow: inset 0 2px 4px rgba(0,0,0,0.3); /* inner shadow */
box-shadow: 0 0 0 3px blue; /* spread only = outline effect */
}

/* text-shadow: offset-x offset-y blur color */
h1 {
text-shadow: 2px 2px 4px rgba(0,0,0,0.3);
text-shadow: 0 0 10px #fff, 0 0 20px #fff, 0 0 40px #ff00de; /* glow */
}

```

5.8 Filters & Opacity

```

.image {
opacity: 0.7; /* 0=invisible, 1=fully visible – affects entire element+children
*/

/* CSS filter functions */
filter: blur(4px);
filter: brightness(0.8); /* < 1 = darker, > 1 = brighter */
filter: contrast(1.5);
filter: grayscale(100%);
filter: sepia(50%);
filter: saturate(2);
filter: hue-rotate(90deg);
filter: invert(100%);
filter: drop-shadow(4px 4px 8px black); /* respects transparency, unlike
box-shadow */

/* Stack multiple filters */
filter: brightness(1.1) contrast(1.2) saturate(1.3);

/* backdrop-filter – blur behind the element (frosted glass) */
backdrop-filter: blur(10px);
background: rgba(255,255,255,0.2); /* needs partial transparency to see effect */
}

```


Flexbox

One-dimensional layout for rows or columns. The most-used layout tool in modern CSS.

6.1 The Mental Model

Flexbox works on a **parent-child relationship**. The parent is the **flex container**; the direct children become **flex items**. You control layout from the container, and fine-tune individual items on the items themselves. Flexbox works along one axis at a time: either a row (horizontal) or a column (vertical). The axis you choose is the **main axis**; the perpendicular axis is the **cross axis**.

```
.container {  
  display: flex; /* or display: inline-flex */  
}
```

6.2 Container Properties

flex-direction — sets the main axis

```
.container {  
  flex-direction: row; /* → default: left to right */  
  flex-direction: row-reverse; /* ← right to left */  
  flex-direction: column; /* ↓ top to bottom */  
  flex-direction: column-reverse; /* ↑ bottom to top */  
}
```

flex-wrap — should items wrap to next line?

```
.container {
flex-wrap: nowrap; /* default – force all on one line, may overflow */
flex-wrap: wrap; /* items wrap to new lines when needed */
flex-wrap: wrap-reverse; /* wrap but in reverse cross-axis direction */
}

/* Shorthand: flex-flow = flex-direction flex-wrap */
.container { flex-flow: row wrap; }
```

justify-content — aligns items along the MAIN axis

```
.container {
justify-content: flex-start; /* default: pack to start */
justify-content: flex-end; /* pack to end */
justify-content: center; /* center */
justify-content: space-between; /* equal gaps BETWEEN items, no outer gaps */
justify-content: space-around; /* equal gaps around each item (outer = half) */
justify-content: space-evenly; /* equal gaps everywhere including outer */
}
```

align-items — aligns items along the CROSS axis (single line)

```
.container {
align-items: stretch; /* default: fill cross axis height/width */
align-items: flex-start; /* align to start of cross axis */
align-items: flex-end; /* align to end of cross axis */
align-items: center; /* center on cross axis */
align-items: baseline; /* align text baselines */
}
```

align-content — aligns MULTIPLE LINES on the cross axis (only with flex-wrap)

```
.container {
flex-wrap: wrap; /* must have wrapping for this to work */
align-content: stretch; /* default */
align-content: flex-start;
align-content: flex-end;
align-content: center;
align-content: space-between;
align-content: space-around;
align-content: space-evenly;
}
```

gap — space between flex items

```
.container {
gap: 16px; /* equal gap on both axes */
gap: 20px 10px; /* row-gap column-gap */
row-gap: 20px;
column-gap: 10px;
}
```

6.3 Flex Item Properties

flex-grow — how much should an item expand to fill available space?

```
/* flex-grow: <number> — default 0 (don't grow) */

.item-a { flex-grow: 1; } /* takes 1 share of available space */
.item-b { flex-grow: 2; } /* takes 2 shares (twice as much as item-a) */
.item-c { flex-grow: 0; } /* does not grow at all */

/* Making all items equal width: */
.item { flex-grow: 1; } /* all grow equally → equal width */
```

flex-shrink — how much should an item shrink when space is tight?

```
/* flex-shrink: <number> – default 1 (shrink proportionally) */  
  
.item { flex-shrink: 1; } /* default: can shrink */  
.item { flex-shrink: 0; } /* never shrink below flex-basis */  
.item { flex-shrink: 3; } /* shrinks 3× as fast as flex-shrink:1 items */
```

flex-basis — the initial size before growing/shrinking

```
/* flex-basis: <length> | auto | content */  
  
.item { flex-basis: auto; } /* default: use width/height */  
.item { flex-basis: 200px; } /* start at 200px */  
.item { flex-basis: 33.33%; } /* start at 1/3 of container */  
.item { flex-basis: 0; } /* start at nothing, let grow handle size */
```

flex — the shorthand (use this 99% of the time)

```
/* flex: flex-grow flex-shrink flex-basis */  
  
.item { flex: 1; } /* 1 1 0% – grow, shrink, start from 0 */  
.item { flex: 2; } /* 2 1 0% – grows twice as fast */  
.item { flex: auto; } /* 1 1 auto – natural size, grow, shrink */  
.item { flex: none; } /* 0 0 auto – fixed natural size, won't flex */  
.item { flex: 0 0 200px; } /* fixed 200px, no grow or shrink */  
  
/* Common: equal-width columns */  
.col { flex: 1; }  
  
/* Common: fixed sidebar, flexible content */  
.sidebar { flex: 0 0 280px; }  
.content { flex: 1; }
```

order — reorder visually without changing HTML

```
/* Default order is 0. Lower = appears first. */  
.first { order: -1; } /* move to beginning */  
.last { order: 1; } /* move to end */  
.normal { order: 0; } /* default position */  
  
/* Note: visual only. Screen reader order = DOM order */
```

align-self — override align-items for one specific item

```
.special-item {  
  align-self: flex-start; /* overrides container's align-items */  
  align-self: center;  
  align-self: flex-end;  
  align-self: stretch;  
  align-self: auto; /* default: inherits align-items */  
}
```

6.4 Common Flexbox Patterns

```

/* 1. Perfect centering – vertically & horizontally */
.center {
display: flex;
justify-content: center;
align-items: center;
}

/* 2. Navigation bar with logo left, links right */
.navbar {
display: flex;
justify-content: space-between;
align-items: center;
}

/* 3. Sticky footer – push footer to bottom */
body {
display: flex;
flex-direction: column;
min-height: 100vh;
}
main { flex: 1; }

/* 4. Card grid with wrapping */
.card-grid {
display: flex;
flex-wrap: wrap;
gap: 24px;
}
.card {
flex: 1 1 280px; /* grow, shrink, min-width 280px */
}

/* 5. Fixed sidebar + fluid content */
.layout {
display: flex;

gap: 32px;
}
.sidebar { flex: 0 0 260px; }
.content { flex: 1; min-width: 0; } /* min-width:0 prevents overflow */

```

■ The most important flex pattern to memorize: `display:flex + justify-content:center + align-items:center` = perfect centering. Works for almost any centering need.

CSS Grid

Two-dimensional layout — rows AND columns simultaneously. The most powerful CSS layout system.

7.1 The Mental Model

Grid works in two dimensions simultaneously — you define **tracks** (rows and columns), creating a grid of **cells**. Items can span multiple cells. The container defines the structure; items are placed into it either automatically or explicitly. Grid shines for macro page layouts and any 2D structure like image galleries, dashboards, and magazine layouts.

7.2 Defining the Grid — Container Properties

[grid-template-columns & grid-template-rows](#)

```

.container {
display: grid;

/* 3 fixed-width columns */
grid-template-columns: 200px 200px 200px;

/* 3 equal flexible columns using fr (fraction unit) */
grid-template-columns: 1fr 1fr 1fr;

/* Sidebar layout: fixed left, flexible right */
grid-template-columns: 260px 1fr;

/* Holy grail: fixed sides, flexible center */
grid-template-columns: 200px 1fr 200px;

/* Mix of units */
grid-template-columns: 1fr 2fr minmax(200px, 1fr);

/* Defining rows */
grid-template-rows: 80px 1fr 60px; /* header body footer heights */
grid-template-rows: auto 1fr auto; /* auto = content height */
}

```

repeat() — avoid repeating yourself

```

.container {
/* These are equivalent: */
grid-template-columns: 1fr 1fr 1fr 1fr;
grid-template-columns: repeat(4, 1fr);

/* Mixed: 2 fixed + 3 flexible */
grid-template-columns: repeat(2, 200px) repeat(3, 1fr);

/* auto-fill: create as many tracks as fit */
grid-template-columns: repeat(auto-fill, 200px);

/* auto-fit: like auto-fill but collapses empty tracks */
grid-template-columns: repeat(auto-fit, 200px);

/* Responsive grid – columns adapt, no media queries needed! */
grid-template-columns: repeat(auto-fit, minmax(280px, 1fr));
}

```

■ `repeat(auto-fit, minmax(280px, 1fr))` is one of the most powerful single lines in CSS — creates a fully responsive grid with no media queries. Cards will arrange themselves optimally.

gap

```
.container {  
  gap: 24px; /* equal row and column gap */  
  gap: 32px 16px; /* row-gap column-gap */  
  row-gap: 32px;  
  column-gap: 16px;  
}
```

Implicit grid — what happens when items overflow

```
.container {  
  grid-template-columns: repeat(3, 1fr);  
  /* Only columns defined – browser creates rows automatically */  
  /* Control implicit row size */  
  grid-auto-rows: 200px; /* fixed */  
  grid-auto-rows: minmax(100px, auto); /* at least 100px, grows to fit */  
  /* Direction of auto-placement */  
  grid-auto-flow: row; /* default: fill rows */  
  grid-auto-flow: column; /* fill columns */  
  grid-auto-flow: row dense; /* fill gaps with smaller items */  
}
```

7.3 Placing Items — Grid Lines

Grid lines are numbered starting from 1. A 3-column grid has 4 vertical lines (1, 2, 3, 4). You can also count from the end using negative numbers (-1 = last line).

```
.item {  
  /* Column placement: start-line / end-line */  
  grid-column: 1 / 3; /* from line 1 to line 3 (spans 2 columns) */  
  grid-column: 1 / -1; /* from first to last line = full width */  
  grid-column: 2 / span 2; /* start at line 2, span 2 columns */  
  grid-column: span 3; /* span 3 columns from wherever auto-placed */  
  
  /* Row placement */  
  grid-row: 1 / 2;  
  grid-row: 1 / span 2;  
  
  /* Shorthand: grid-area = row-start / col-start / row-end / col-end */  
  grid-area: 1 / 1 / 3 / 4;  
}
```

7.4 Named Areas — grid-template-areas

The most readable way to define layout. Draw the grid visually in your CSS:

```

.container {
display: grid;
grid-template-columns: 200px 1fr 200px;
grid-template-rows: 80px 1fr 60px;
gap: 16px;
min-height: 100vh;

grid-template-areas:
"header header header"
"sidebar content aside"
"footer footer footer";
/* Each string = a row. Each word = an area name. */
/* Use . for an empty cell: "sidebar . aside" */
}

/* Assign elements to areas by name */
header { grid-area: header; }
.sidebar{ grid-area: sidebar; }
.content{ grid-area: content; }
aside { grid-area: aside; }
footer { grid-area: footer; }

```

Named lines

```

.container {
grid-template-columns:
[sidebar-start] 200px
[sidebar-end content-start] 1fr
[content-end];
}

.content { grid-column: content-start / content-end; }

```

7.5 Item Alignment in Grid

```

/* Container: align ALL items */
.container {
/* Along row axis (horizontal) */
justify-items: start | end | center | stretch; /* default: stretch */

/* Along column axis (vertical) */
align-items: start | end | center | stretch;

/* Align the entire grid within the container (if grid < container) */
justify-content: start | end | center | space-between | space-around |
space-evenly;
align-content: start | end | center | space-between | space-around | space-evenly;

/* Shorthand */
place-items: center; /* align-items + justify-items */
place-items: start center; /* align justify */
place-content: center;
}

/* Individual item overrides */
.item {
justify-self: start | end | center | stretch;
align-self: start | end | center | stretch;
place-self: center; /* shorthand */
}

```

7.6 Grid vs Flexbox — When to Use Which

```

/* Use FLEXBOX when you have a 1D sequence of items */
.nav { display: flex; gap: 16px; align-items: center; }

/* Use GRID when you have 2D structure to control */
.page { display: grid; grid-template-areas: "header" "main" "footer"; }

/* They work TOGETHER: grid for page structure, flex inside grid cells */
.card-grid { display: grid; grid-template-columns: repeat(auto-fit,
minmax(280px,1fr)); }

.card { display: flex; flex-direction: column; } /* flex inside grid */

```

■ Grid and Flexbox are not competitors — they are complementary. Use Grid for the macro layout of the page, Flexbox for the micro layout within components.

Transitions & Animations

Add motion to your UI. Transforms, transitions, @keyframes, and performance.

8.1 transform — change shape without affecting layout

The **transform** property moves, rotates, scales, or skews an element visually. Critically, transforms do **not** affect document flow — surrounding elements don't shift.

```

.box {
  /* translate: move element */
  transform: translateX(50px); /* move right 50px */
  transform: translateY(-20px); /* move up 20px */
  transform: translate(50px, -20px); /* X and Y together */
  transform: translate(50%, -50%); /* % relative to element size */

  /* rotate */
  transform: rotate(45deg); /* clockwise 45 degrees */
  transform: rotate(-180deg); /* counter-clockwise */
  transform: rotate(0.25turn); /* quarter turn */

  /* scale */
  transform: scale(1.5); /* 150% of original size, both axes */
  transform: scale(2, 0.5); /* double width, half height */
  transform: scaleX(1.2);
  transform: scaleY(0.8);

  /* skew - slant */
  transform: skewX(15deg);
  transform: skewY(10deg);
  transform: skew(15deg, 10deg);

  /* Chain multiple transforms (space-separated, applied right to left) */
  transform: translateY(-50%) rotate(45deg) scale(1.1);
}

```

transform-origin — pivot point for transforms

```
.box {
transform-origin: center center; /* default — center of element */
transform-origin: top left;
transform-origin: 0 0;
transform-origin: 50% 50%;
transform-origin: bottom right;
transform-origin: 20px 30px;

/* Example: rotate around top-left corner */
transform-origin: top left;
transform: rotate(45deg);
}
```

3D Transforms

```
.scene {
perspective: 800px; /* REQUIRED on PARENT for 3D effect */
/* Smaller = more dramatic 3D, Larger = more subtle */
}

.box {
transform: rotateX(45deg); /* tilt toward viewer */
transform: rotateY(45deg); /* spin left/right */
transform: rotateZ(45deg); /* same as rotate(45deg) */
transform: rotate3d(1, 1, 0, 45deg); /* axis vector + angle */

transform: translateZ(100px); /* move toward/away from viewer */
transform: translate3d(0, 0, 100px);

transform: perspective(800px) rotateY(45deg); /* per-element perspective */
transform-style: preserve-3d; /* children exist in 3D space */
backface-visibility: hidden; /* hide back of element when rotated */
}
```

8.2 transition — animate between two states

Transitions automatically animate a property change — from its current value to a new value — smoothly over a specified duration.

```

.button {
background: blue;
transform: scale(1);

/* transition: property duration timing-function delay */
transition: background 0.3s ease;
transition: transform 0.2s ease-out;

/* Multiple transitions (comma-separated) */
transition:
background 0.3s ease,
transform 0.2s ease-out,
box-shadow 0.3s ease;

/* transition: all – transitions every changing property (avoid for perf) */
transition: all 0.3s ease;
}

.button:hover {
background: darkblue;
transform: scale(1.05);
box-shadow: 0 8px 25px rgba(0,0,0,0.2);
}

```

Timing Functions

```

/* Keywords */
transition-timing-function: ease; /* default: slow start, fast middle, slow end
*/
transition-timing-function: linear; /* constant speed */
transition-timing-function: ease-in; /* slow start, fast end (accelerate) */
transition-timing-function: ease-out; /* fast start, slow end (decelerate) */
transition-timing-function: ease-in-out; /* slow start and end */
transition-timing-function: step-start; /* jump instantly at start */
transition-timing-function: step-end; /* jump instantly at end */
transition-timing-function: steps(4, end); /* 4 discrete steps */

/* cubic-bezier for custom curves – use cubic-bezier.com */
transition-timing-function: cubic-bezier(0.68, -0.55, 0.265, 1.55); /* bounce */
transition-timing-function: cubic-bezier(0.4, 0, 0.2, 1); /* Material Design */

```

8.3 @keyframes — define an animation sequence

```
/* Define the animation with a name */
@keyframes fadeIn {
  from { opacity: 0; }
  to { opacity: 1; }
}

/* Use percentage stops for more control */
@keyframes slideUp {
  0% { transform: translateY(40px); opacity: 0; }
  60% { transform: translateY(-5px); opacity: 1; }
  100% { transform: translateY(0); }
}

/* Multiple properties */
@keyframes pulse {
  0%, 100% { transform: scale(1); box-shadow: 0 0 0 0 rgba(0,0,255,0.4); }
  50% { transform: scale(1.05); box-shadow: 0 0 0 10px rgba(0,0,255,0); }
}

/* Spinning loader */
@keyframes spin {
  from { transform: rotate(0deg); }
  to { transform: rotate(360deg); }
}
```

8.4 animation — applying @keyframes

```

.element {
  /* animation: name duration timing delay iteration direction fill-mode play-state */
  /*
  animation: fadeIn 0.5s ease 0s 1 normal forwards running;
  */
  /* Individual properties */
  animation-name: fadeIn;
  animation-duration: 0.5s;
  animation-timing-function: ease;
  animation-delay: 0.2s; /* wait 0.2s before starting */
  animation-iteration-count: 1; /* number of times | infinite */
  animation-direction: normal;
  /* normal | reverse | alternate | alternate-reverse */
  animation-fill-mode: none;
  /* none | forwards | backwards | both */
  animation-play-state: running; /* running | paused (pause with JS!) */
}

```

animation-fill-mode — critical for beginning and end states

```

/* none – default: returns to original state before/after */
/* forwards – stays at final keyframe state after animation ends */
/* backwards – jumps to first keyframe state during the delay period */
/* both – applies both forwards and backwards */

/* Common: animate in and stay */
.fade-in {
  opacity: 0;
  animation: fadeIn 0.5s ease forwards;
  /* forwards = stays visible after animation completes */
}

/* Staggered animation with delay */
.item:nth-child(1) { animation-delay: 0s; }
.item:nth-child(2) { animation-delay: 0.1s; }
.item:nth-child(3) { animation-delay: 0.2s; }
.item:nth-child(4) { animation-delay: 0.3s; }

```

Multiple animations

```

.element {
  /* Comma-separated list runs all animations simultaneously */
  animation:
    fadeIn 0.5s ease forwards,
    slideUp 0.5s ease forwards,
    pulse 2s ease-in-out 0.5s infinite;
}

```

8.5 Performance — What to Animate

Not all CSS properties are equal in performance. The browser has to do different amounts of work depending on what changes:

Property	Values	What it does
transform	<i>GPU-accelerated, no layout recalc</i>	■ Always prefer. translateX/Y, scale, rotate — all composited on GPU.
opacity	<i>GPU-accelerated</i>	■ Very fast. Pure compositing, no layout or paint.
color, background, border-color, box-shadow	<i>Paint only — no layout</i>	■■ OK for simple cases, avoid animating many elements at once.
width, height, margin, padding, top, left	<i>Triggers layout (reflow)</i>	■ Avoid animating. Causes the browser to recalculate all affected elements.
font-size	<i>Triggers layout + paint</i>	■ Avoid. Use transform: scale instead for size animations.

```
/* ■ BAD – triggers layout on every frame */
@keyframes bad { from { width: 100px; } to { width: 200px; } }

/* ■ GOOD – GPU-composited */
@keyframes good { from { transform: scaleX(0.5); } to { transform: scaleX(1); } }

/* will-change: hint to browser to promote element to its own layer */
.animated {
  will-change: transform, opacity;
  /* Use ONLY on elements you KNOW will animate. Overuse wastes GPU memory. */
}
```

8.6 Responsive Design & Media Queries


```

/* Syntax */
@media (max-width: 768px) {
  /* styles for screens 768px wide or narrower */
  .sidebar { display: none; }
  .content { width: 100%; }
}

/* Mobile-first approach (PREFERRED) */
/* Write base styles for mobile, then ADD styles for larger screens */
.card { flex-direction: column; } /* mobile: stack vertically */
@media (min-width: 768px) {
  .card { flex-direction: row; } /* tablet+: side by side */
}
@media (min-width: 1024px) {
  .card { max-width: 800px; margin: 0 auto; }
}

/* Common breakpoints */
@media (max-width: 480px) { /* phone */ }
@media (max-width: 768px) { /* tablet portrait */ }
@media (max-width: 1024px) { /* tablet landscape / small laptop */ }
@media (max-width: 1280px) { /* laptop */ }

/* Multiple conditions */
@media (min-width: 600px) and (max-width: 900px) { }

/* Media features */
@media (orientation: landscape) { }
@media (prefers-color-scheme: dark) {
  :root {
    --color-bg: #0a0a0a;
    --color-text: #f0f0f0;
  }
}
@media (prefers-reduced-motion: reduce) {

```

```
*, *::before, *::after {
  animation-duration: 0.01ms !important;
  transition-duration: 0.01ms !important;
}
}
@media print {
  nav, .ads, .sidebar { display: none; }
  body { font-size: 12pt; color: black; }
}
```

8.7 ::before and ::after in Depth

Pseudo-elements create virtual child elements at the start or end of any element. They require **content: ""** to render. They are extraordinarily versatile for decorative effects without extra HTML.

```

/* Basic structure */
.element::before {
content: ""; /* required — even if empty */
display: block; /* or inline, flex, etc. */
/* now treat like any other element: size, position, color... */
}

/* Icon/bullet prefix */
.check-item::before {
content: "✓";
color: green;
margin-right: 8px;
}

/* Tooltip via attr() */
[data-tooltip]::after {
content: attr(data-tooltip); /* reads the data-tooltip HTML attribute! */
position: absolute;
background: black;
color: white;
padding: 4px 8px;
border-radius: 4px;
font-size: 0.875rem;
white-space: nowrap;
bottom: 100%;
left: 50%;
transform: translateX(-50%);
pointer-events: none;
opacity: 0;
transition: opacity 0.2s;
}
[data-tooltip]:hover::after { opacity: 1; }

/* Overlay on card hover */
.card {

```

```

position: relative;
overflow: hidden;
}
.card::after {
content: "";
position: absolute;
inset: 0;
background: rgba(0,0,0,0);
transition: background 0.3s ease;
}
.card:hover::after {
background: rgba(0,0,0,0.4);
}

/* Decorative underline on heading */
h2 { position: relative; display: inline-block; }
h2::after {
content: "";
position: absolute;
bottom: -4px;
left: 0;
width: 0;
height: 3px;
background: var(--color-primary);
transition: width 0.4s ease;
}
h2:hover::after { width: 100%; }

/* Clearfix (legacy) */
.clearfix::after {
content: "";
display: table;
clear: both;
}

```

■ `::before` and `::after` are always the first and last children of their element in the visual rendering. Use `position: absolute` on them (with `position: relative` on the parent) for most decorative uses.

Congratulations. You now have a complete, professional-grade understanding of HTML and CSS — from the foundational document structure all the way through the most powerful layout systems (Flexbox and Grid) and animation capabilities the platform offers. The path forward: build obsessively, read the MDN Web Docs daily, inspect the CSS of sites you admire, and remember — every expert was once a beginner staring at a blank page.